

Дообучение LLM моделей

Continued Pretraining (CPT)

Смысл. Продолжаем предобучение на доменном корпусе без разметки (у Causal LM — предсказываем следующий токен). Синонимы: **DAPT** (Domain-Adaptive), **TAPT** (Task-Adaptive).

- **Данные:** неразмеченный текст домена (юридический, медиц., HR-документы).
- **Лосс:** для авто-регрессии

$$\mathcal{L}_{\text{CPT}}(\theta) = - \sum_t \log p_{\theta}(x_t \mid x_{<t})$$

- **Когда:** надо «вливать словарь/стилистику» домена перед SFT.
- **Плюсы:** дешёвые данные, хорошо улучшает понимание терминов.
- **Минусы:** не учит формату ответа/поведению.
- **Пример (HR):** продолжаем предобучение на описаниях вакансий и резюме → модель лучше «чувствует» роли, грейды.

SFT — Supervised Fine-Tuning

Смысл. Учим модель воспроизводить эталонные ответы на размеченных парах *запрос* → *ответ*.

- **Данные:** пары (*prompt*, *ideal response*) в чат-шаблоне.
- **Лосс** (кросс-энтропия):

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{t=1}^T \log p_{\theta}(y_t \mid x, y_{<t})$$

- **Когда:** нужен конкретный формат/стиль ответов, доменная лексика.
- **Плюсы:** просто, стабильно, воспроизводимо.
- **Минусы:** не учит «что лучше», не даёт отказов/тонирование поведения.

Preference-tuning (настройка по предпочтениям)

RLHF (PPO)

Три шага: (1) SFT-policy; (2) Reward-model по парам (*chosen, rejected*); (3) PPO с KL-штрафом к референсу.

- Лосс (идея):

$$\max_{\theta} \mathbb{E}[r_{\phi}(x, y)] - \beta D_{\text{KL}}(\pi_{\theta}(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x))$$

- Плюсы: гибко, можно сложные reward'ы (полезность, безопасность).
- Минусы: сложнее и дороже (RM+PPO, стабильность).

RLHF (PPO). Как работает.

1. Берём prompt x .
2. Политика π_θ генерирует ответ $y = (a_1, \dots, a_T)$.
3. Считаем **последовательностную** награду:

$$R = r_\phi(x, y) - \beta \cdot (\log \pi_\theta(y \mid x) - \log \pi_{\text{ref}}(y \mid x)),$$

где $\log \pi(y \mid x) = \sum_t \log \pi(a_t \mid s_t)$.

Иногда добавляют **штраф за длину**, чтобы модель не «хакала» RM болтовнёй.

4. Раскладываем R по токенам (например, весь R — в хвостовой токен EOS, либо равномерно), считаем **advantage** A_t (через GAE).
5. Делаем несколько **PPO-эпох** обновления θ и ψ на этом батче.

DPO — Direct Preference Optimization

Обходит RM/PPO, оптимизирует лог-шансы выбранного ответа относительно отвергнутого, с «вшитым» KL.

- Лосс (упрощённо):

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\left(\beta \left[\log \pi_{\theta}(y^{+} | x) - \log \pi_{\theta}(y^{-} | x) - \log \pi_{\text{ref}}(y^{+} | x) + \log \pi_{\text{ref}}(y^{-} | x)\right]\right)$$

- Плюсы: проще RLHF, меньше подвижных частей.
- Минусы: нужны аккуратные парные данные.

PEFT — Parameter-Efficient Fine-Tuning

Prefix-/P-Tuning

Обучаем «виртуальные токены» (эмбединги) в начале последовательности.

- **Память:** минимальная.
- **Влияние:** не меняет граф, но **съедает контекст** и удлиняет префикл:

$$L_{\text{eff}} = L_{\text{max}} - (l_p + l_i).$$

- **Подходит:** быстрые прототипы, когда важно «вообще не трогать» веса.

Prefix-/P-Tuning как работает

Пусть:

- вход $x = (x_1, \dots, x_n)$, эмбединги $E(x) \in \mathbb{R}^{n \times d}$,
- мягкий префикс $P \in \mathbb{R}^{m \times d}$ — только его мы обучаем.

Мы подаём в модель не $E(x)$, а **конкатенацию**

$$Z = [P; E(x)] \in \mathbb{R}^{(m+n) \times d}.$$

Дальше обычный трансформер: на каждом слое

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Просто теперь K, V содержат и префиксные позиции. Итог: модель **видит** перед задачей специальные векторы и подстраивает ответы.

Функция потерь стандартная (как в SFT):

$$\mathcal{L}_{\text{SFT}}(\theta, P) = - \sum_{t=1}^T \log p_{\theta, P}(y_t \mid [P; x], y_{<t}),$$

где θ — замороженные веса модели, обучаем только P .

Adapters

Вставляем узкие MLP-вставки (бутылочные) в блоки трансформера.

- **Плюсы:** доменная изоляция, просто коммутировать адаптеры.
- **Минус:** вставки остаются на инференсе → +латентность.

Мы не трогаем веса трансформера, а **встраиваем в каждый блок маленькую «бутылочную» вставку** (узкий MLP). Она учится на наших данных и добавляет к исходным признакам маленькую «поправку».

На инференсе эта вставка **остаётся** в графе → даёт немного **доп. задержки**.

Adapters в архитектуре LLM

Классические варианты расстановки:

- **Houlsby**: два адаптера в блоке — после MHA и после FFN.
- **Pfeiffer**: один адаптер в FFN-подблоке (иногда «параллельно» слою).

Схема для одного места вставки:

$$\boxed{h' = h + f_{\text{adapter}}(h)}, \quad f_{\text{adapter}}(h) = s \cdot W_{\uparrow} \sigma(W_{\downarrow} h)$$

- $h \in \mathbb{R}^d$ — вход в точке вставки (после слоя/нормализации).
- $W_{\downarrow} \in \mathbb{R}^{r \times d}$ — сжатие (узкое «горлышко»), $r \ll d$.
- $W_{\uparrow} \in \mathbb{R}^{d \times r}$ — восстановление размерности.
- σ — нелинейность (GELU/ReLU).
- s — маленький скейл (например, $s = 0.1$) → при старте адаптер «почти ноль», сеть ведёт себя как до тюнинга (стабильность).

Обучаем только $W_{\downarrow}$, $W_{\uparrow}$ (и иногда bias/LayerNorm рядом). База θ заморожена.

LoRA

Учите низкоранговое приращение матрицы:

$$W = W_0 + BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k)$$

Число тренируемых весов $\approx r(d + k)$ (в разы меньше $d \cdot k$).

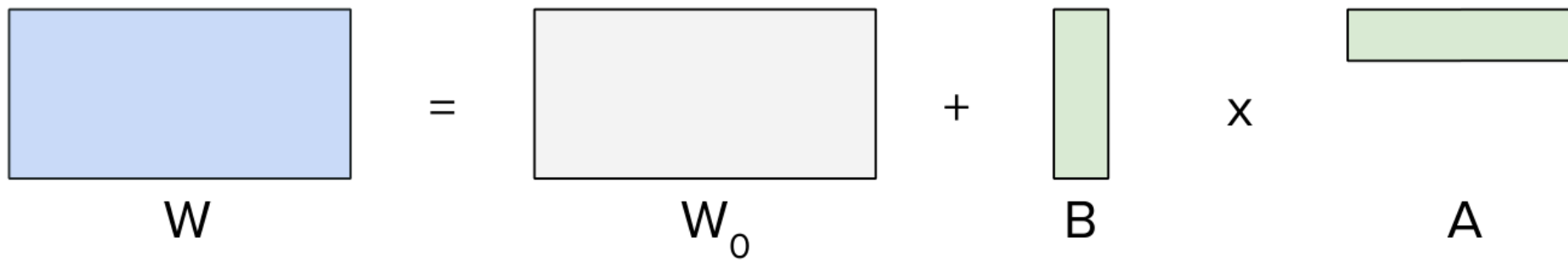
Фишка: можно **merge** BA в $W_0 \rightarrow$ инференс без накладных.

Параметро-эффективная донастройка: LoRA

$W = W_0 + B \cdot A \cdot x$; тренируем A, B (низкий ранг r)

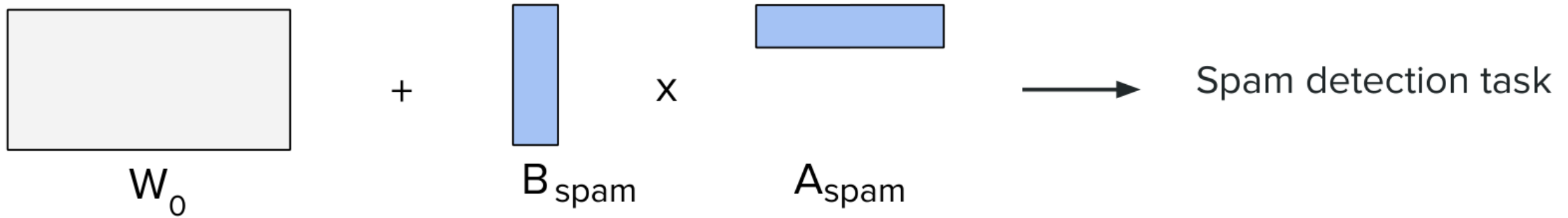
«адаптеры» подключаются под задачи

Idea. Low-Rank Adaptation (LoRA) approximates matrix with product of two low-rank matrices



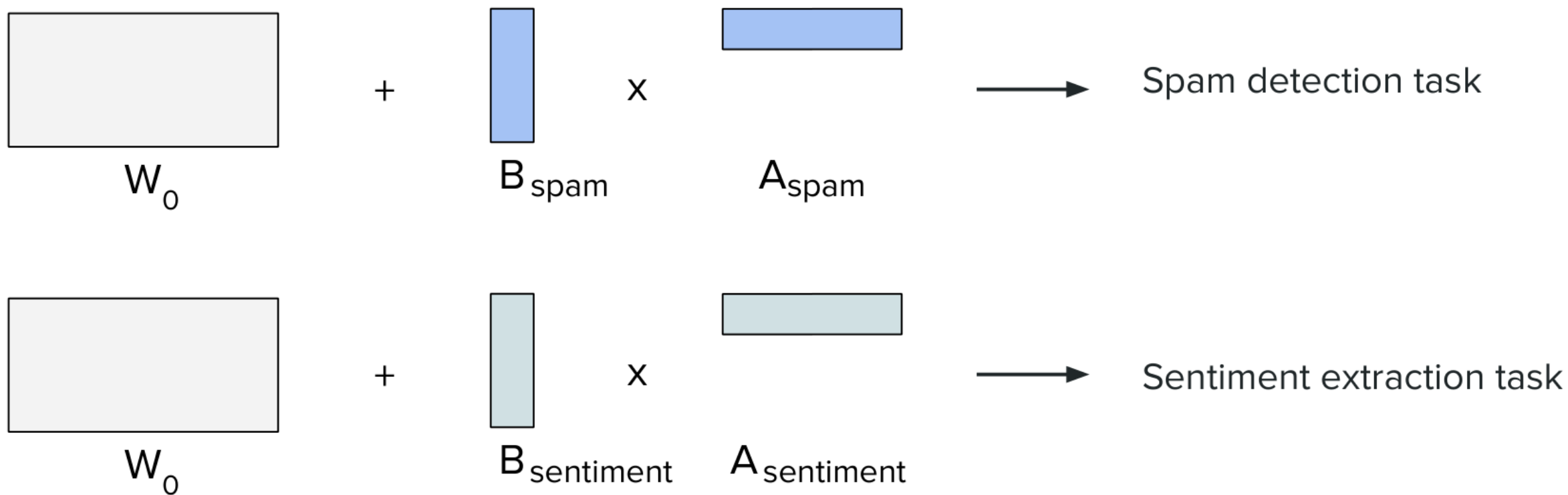
Параметро-эффективная донастройка: LoRA

$W = W_0 + B \cdot A \cdot x$; тренируем A, B (низкий ранг r) «адаптеры» подключаются под задачи



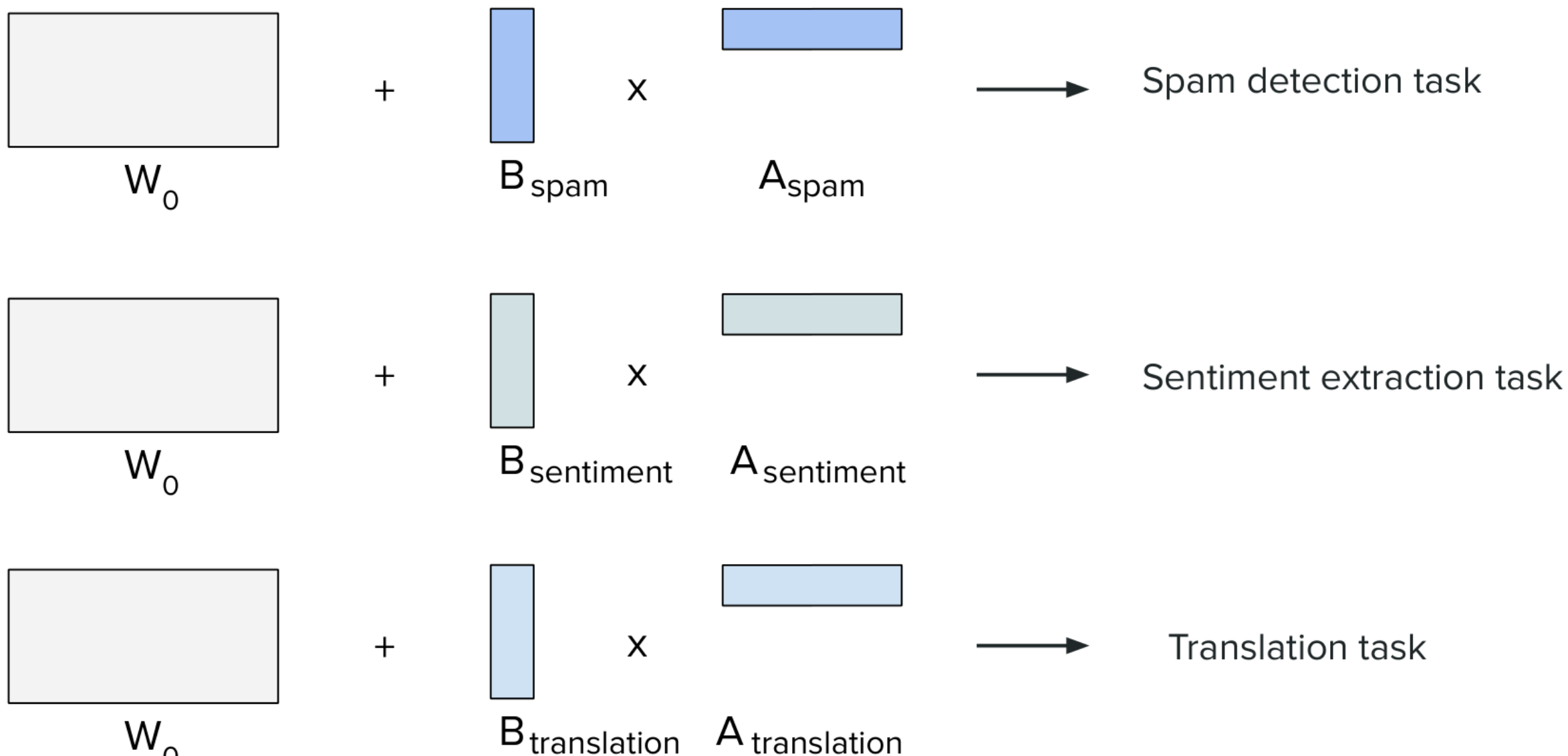
Параметро-эффективная донастройка: LoRA

$W = W_0 + B \cdot A \cdot x$; тренируем A, B (низкий ранг r) «адаптеры» подключаются под задачи



Параметро-эффективная донастройка: LoRA

$W = W_0 + B \cdot A \cdot x$; тренируем A, B (низкий ранг r) «адаптеры» подключаются под задачи

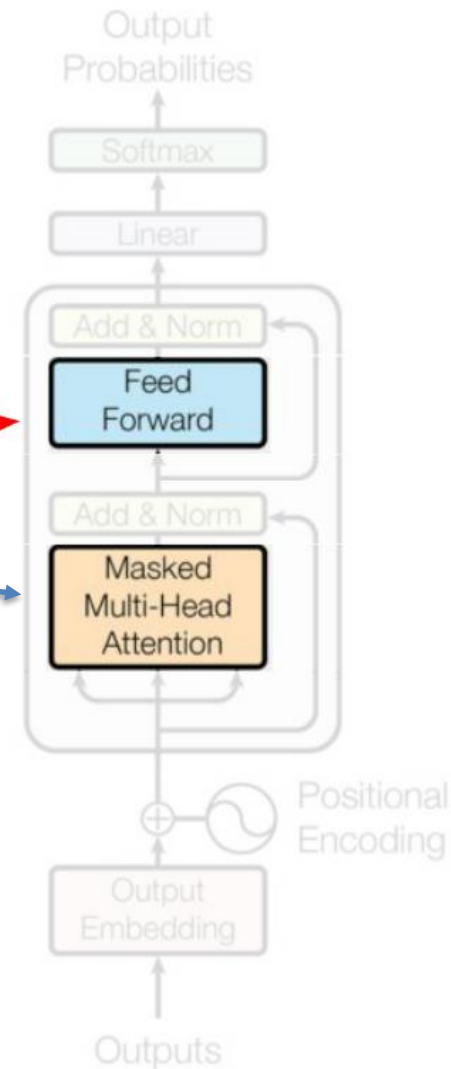


Где применять LoRA

Attention и особенно FFN — современные практики

Баланс ранга r и охвата слоёв

most important location



QLoRA

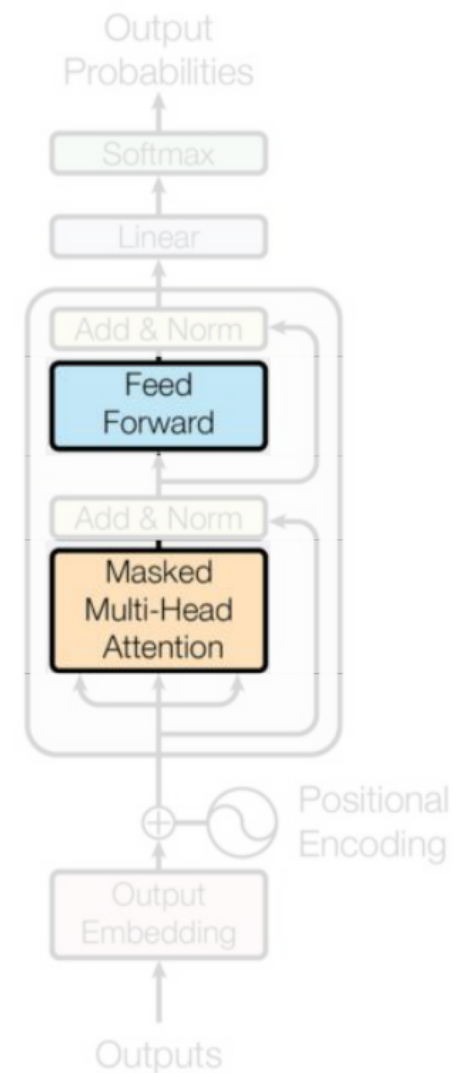
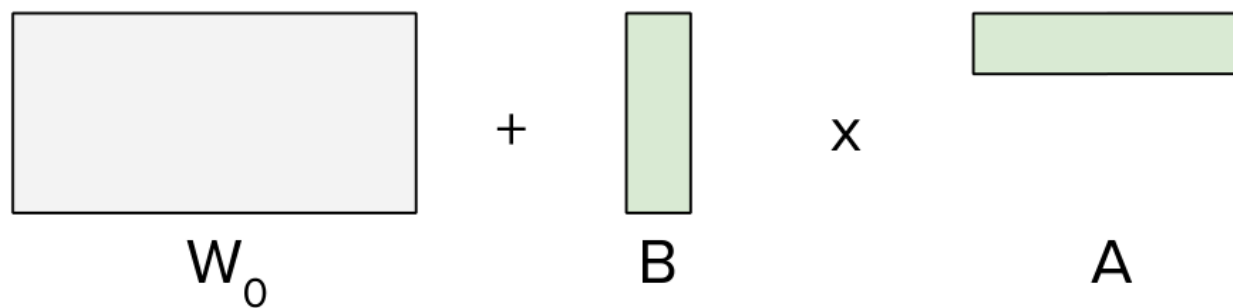
То же LoRA, но база в 4-бит (NF4 + double-quant), адаптеры — в 16-бит.

- **Плюс:** 7–8B модели тюнятся на 1 GPU (16–24 ГБ).
- **Минус:** чуть медленнее из-за 4-бит математики; требователен к настройкам.

QLoRA

Замороженные веса в NF4 + double quant

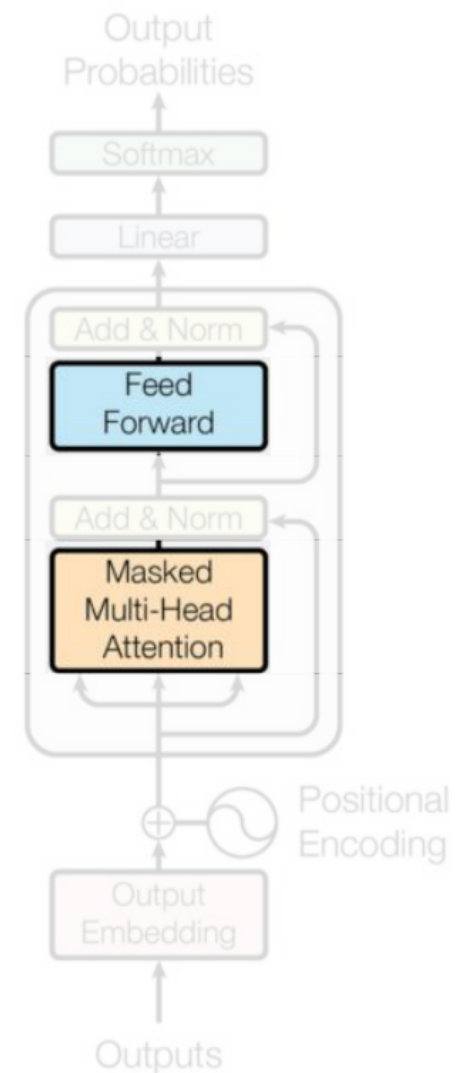
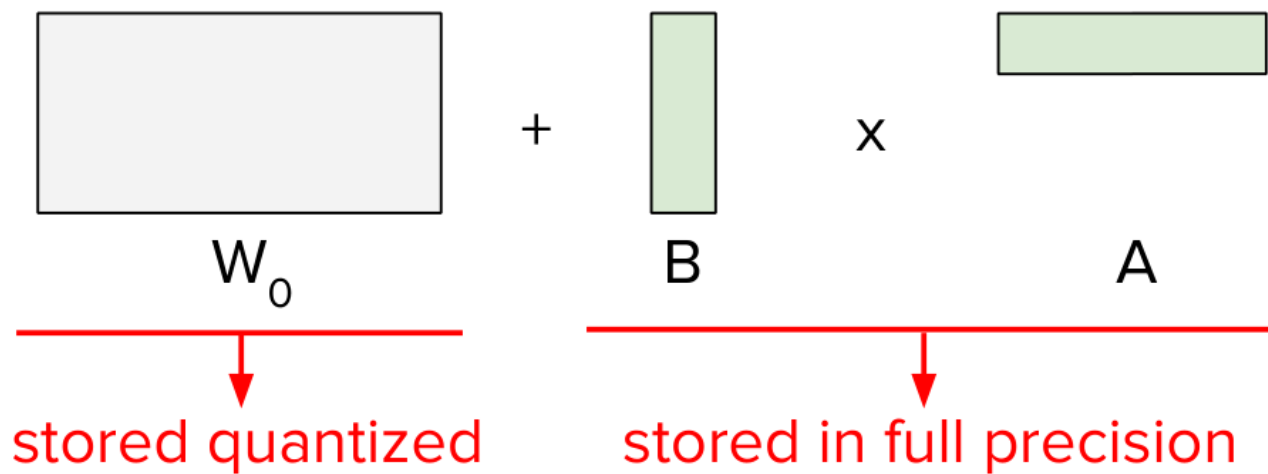
Адаптеры — как в LoRA; экономия VRAM



QLoRA

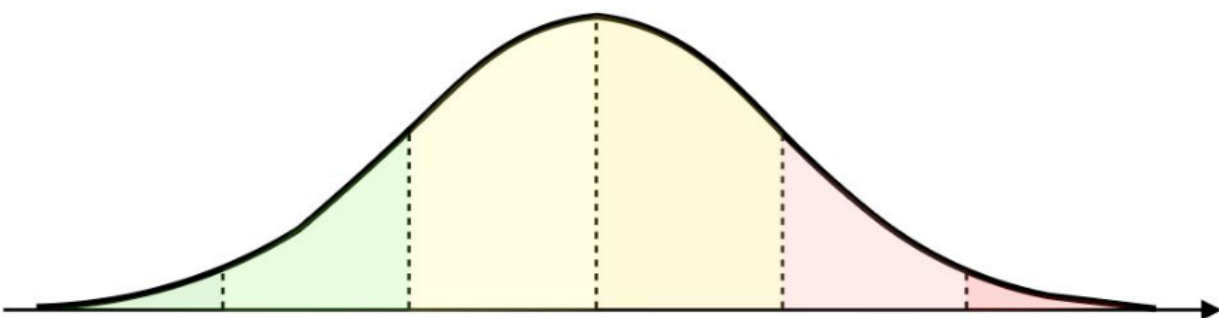
Замороженные веса в NF4 + double quant

Адаптеры — как в LoRA; экономия VRAM



Эффективная квантизация

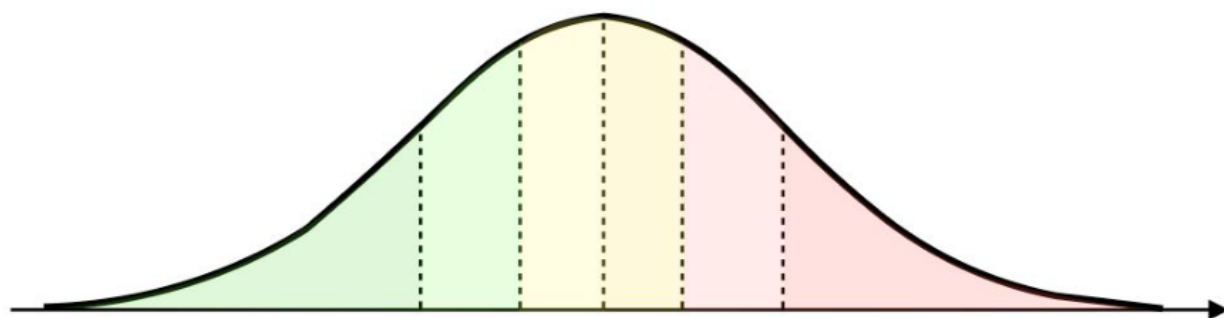
Квантование по квантилям (NF4) эффективнее равномерного
Снижение памяти при близком качестве



×

Uniform distribution cutoffs

INT8



✓

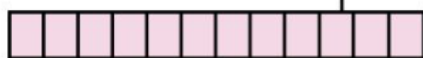
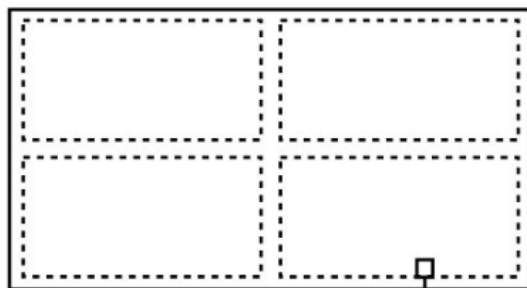
Based on Normal quantiles

NF4

Квантизация...

Квантизация весов/активаций Компромисс точность $\leftrightarrow$ ресурсы

Weights



FP32

Quantization
constants



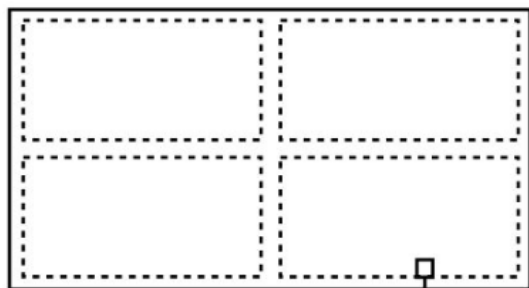
Квантизация...

Квантизация весов/активаций Компромисс точность $\leftrightarrow$ ресурсы

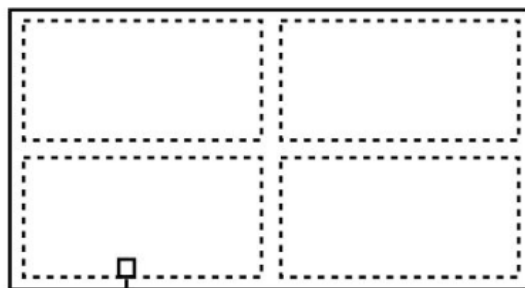
No quantization

Single quantization

Weights

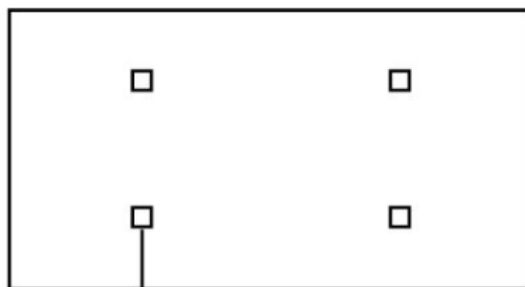


FP32



NF4

Quantization
constants



FP32

Квантизация

...

Квантизация

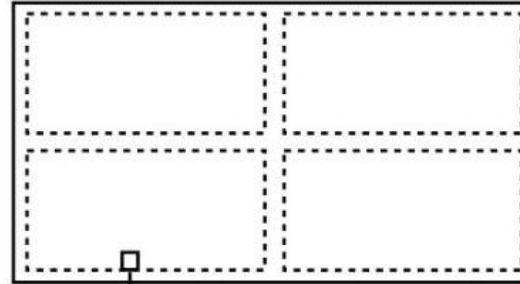
весов/активаций

Компромисс точность

↔ ресурсы

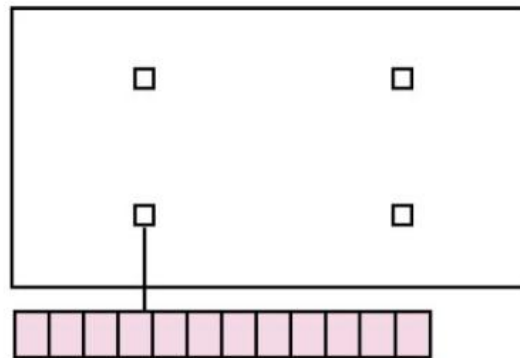
Weights

Single quantization



NF4

Quantization
constants



FP32

QLoRA

Замороженные веса в NF4 + double quant

Адаптеры — в полноточности; экономия VRAM